Module 1 : Sorties

La sortie, ou l'affichage, se fait par défaut sur le périphérique de sortie standard : **L'écran**.

L'opération de sortie se fait en algorithme à l'aide de la procédure **`Écrire(...)`**. **Exemples :**

```
// Affichage d'un message
Ecrire("Votre message")

x ← 4
y ← 7
// Affichage de la valeur d'une variable
Ecrire(x)

// Affichage mixte
Ecrire(x, y)
Ecrire("x = ", x)
```

En Python, elle se fait à l'aide de la fonction **`print(...)`**. Cette fonction offre plusieurs possibilités d'affichage :

- **Affichage d'un message :** Le message doit être entre guillemets doubles **`"message"`** ou entre guillemets simples **`'message'`**. Exemple :

  ```
  print("Votre message")
  print('Votre message')
  ```

- **Affichage de la valeur d'une variable :** Sans guillemets Python affiche la valeur de la variable portant ce nom. Exemple :

  ```
  print(x)
  ```

- **Affichage mixte :** On peut combiner l'affichage de messages et de variables dans une instruction **`print(…)`**. Exemple :

  ```
  # Affichage mixte sur une seule ligne
  print("x =", x)

  # Affichage de variables séparées par des espaces
  x, y, z = 4, 7, 9
  print(x, y, z)

  # Affichage de variables séparées par le symbole ":"
  print(x, y, z, sep=":")

  # Affichage de variables sur une seule ligne séparés par le symbole " - "
  print(x, end=" - ")
  print(y, end=" - ")
  print(z, end=" - ")
  ```

Module 2 : Entrées

L'entrée, ou la saisie, des données utilisateur est effectuée à partir du périphérique d'entrée standard : **Le clavier**.

On utilise, en algorithme, la procédure **`Lire(nom_var)`** pour affecter les entrées de l'utilisateur à la variable **`nom_var`**. **Exemples :**

```
// Saisie d'une chaîne de caractères
Écrire("Votre nom ? ") ; Lire(ch)

// Saisie d'un nombre entier
Écrire("Quel âge as-tu ? ") ; Lire(age)

// Saisie d'un nombre réel
Écrire("Quelle est ta moyenne ? ") ; Lire(moy)
```

En Python, l'entrée des données est effectuée à partir du clavier à l'aide de la fonction **`input(message)`**. Cette fonction retourne toujours une **chaîne de caractères** (type **`str`**). **Exemple :**

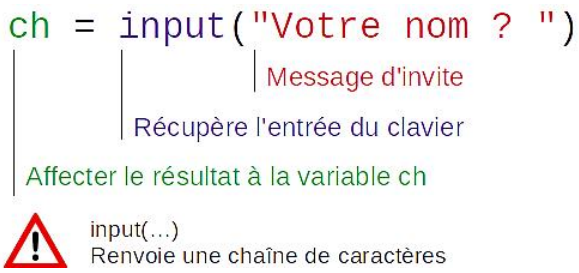

Figure 1: Saisie d'une chaîne de caractères

Souvent, on désire saisir des données numériques de type **entier** ou **réel**. Dans ce cas, il faudra convertir la valeur saisie à l'aide des fonctions :

- **`int(...)`** pour récupérer un **entier**
- Et, **`float(...)`** pour récupérer un **réel**.

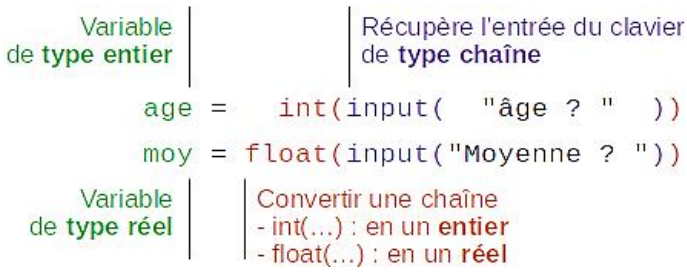

Figure 2: Saisie d'une valeur numérique entier / réel

Module 3 : Structures de données

1. Type caractère

Un caractère peut-être une **lettre alphabétique majuscule** ("A", "B", ..., "Z") ou **minuscule** ("a", "b", ..., "z"), un **chiffre** ("0", "1", ..., "9") ou un **symbole** (" ", "!", "\"", "#", etc).

Chaque caractère possède un code normalisé appelé :

- **Code ASCII** pour les 256 premiers caractères (voir table 1)
- **Code Unicode** au de la des 256 premiers caractères.

Code	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
Car		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
Code	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
Car	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
Code	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79
Car	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Code	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
Car	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
Code	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111
Car	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
Code	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127
Car	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	

Tableau 1: Table des caractères ASCII standard

Fonctions sur les caractères

Fonction	Rôle	Exemple
`ord(car)` `-> entier`	Retrouver le code ASCII/Unicode d'un caractère.	`car ← "A"` `num ← ord(car) // num = 65`
`chr(num)` `-> caractère`	Retrouver le caractère correspondant à un code ASCII/Unicode.	`num ← 64` `car ← chr(num) // car = "@"`

Algorithme
`majus(car)`
`-> caractère`
 	
Python
`car.upper()`
`→ str`
	Retourner la lettre majuscule correspondante si car est une lettre minuscule	`// Algorithme`
		`car1 ← majus("j") // car1 = "J"`
		`car2 ← majus("#") // car2 = "#"`
		`car3 ← majus("F") // car3 = "F"`
		
		`# Python`
		`car1 = "j".upper() // car1 = "J"`
		`car2 = "#".upper() // car2 = "#"`
		`car3 = "F".upper() // car3 = "F"`

Tableau 2: Fonctions sur les caractères

2. Type Chaîne de caractères

Une chaîne de caractères est formée par un ensemble de caractères.

a. Longueur

Une chaîne possède une **longueur** qui indique le nombre de ses caractères. Une chaîne de longueur zéro est appelée **chaîne vide**.

Exemples :

Algorithme	Python
`ch ← "Python"` `l ← Long(ch) // l = 6`	`ch = "Python"` `l = len(ch) # l = 6`

Tableau 3: Longueur d'une chaîne de caractères

b. Concaténation

Une chaîne peut être le résultat de concaténation de deux ou plusieurs chaînes.

Exemples :

Algorithme	Python

`ch1 ← "Py"`
`ch2 ← "thon"`
`ch3 ← ch1 + ch2 // ch3 = "Python"`
	`ch1 = "Py"`
	`ch2 = "thon"`
	`ch3 = ch1 + ch2 # ch3 = "Python"`

Tableau 4: Concaténation de chaînes de caractères

c. Indexation

On peut **accéder aux caractères individuels** d'une chaîne à l'aide de l'opérateur `[]`.

En algorithme, les caractères d'une chaîne sont indexées numériquement. L'indexation commence à partir de 0.

En Python, la chaîne peut-être **indexée** de deux façons différentes.

La figure numéro 3 présente l'exemple d'indexation de la chaîne "Python".

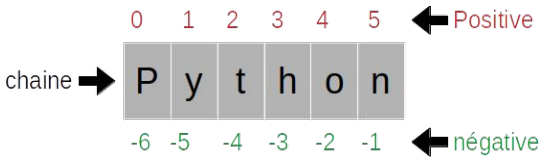

Figure 3: Indexation d'une chaîne de caractères en Python

Exemples :

Algorithme	Python

`ch ← "Python"`
`prem_let ← ch[0] // 1er caractère`
 	
`dern_let ← ch[5] // dernier caractère`
	`ch = "Python"`
	`prem_let = ch[0]  # 1er caractère`
	`prem_let = ch[-6] # 1er caractère`
	`dern_let = ch[5]  # dernier caractère`
	`dern_let = ch[-1] # dernier caractère`

Tableau 5: Indexation d'une chaîne de caractères

d. Sous chaîne

Pour **extraire une partie de la chaîne** en algorithme, on utilise la fonction **`sous_chaine(ch,deb,fin)`**.

L'écriture équivalente en Python est **`ch[deb:fin]`**.

Cette instruction extrait les caractères compris entre les indices **`deb`** inclus et **`fin`** non inclus.

`deb` est l'indice de début, et **`fin`** est l'indice de fin.

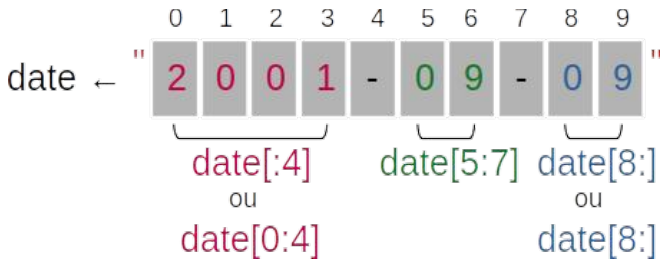

Figure 4: Exemple d'extraction de sous-chaîne

Exemples :

Algorithme	Python

`//  0123456789`
`date ← "2023-09-15"`
`jour ← sous_chaine(date, 8, 10) // "2023"`
`mois ← sous_chaine(date, 5, 7)  // "09"`
`annee ← sous_chaine(date, 0, 4) // "15"`
	`#  0123456789`
	`date = "2023-09-15"`
	`jour = date[8:10] # "2023"`
	`mois = date[5:7] # "09"`
	`annee = date[0:4] # "15"`

`//  1`
`//  01234567890123456789`
`name ← "Mohamed Amine Tounsi"`
`middle_name ← sous_chaine(name, 8, 13)`
	`#  1`
	`#  01234567890123456789`
	`name = "Mohamed Amine Tounsi"`
	`middle_name = name[8:13]`

Tableau 6: Exemples d'extraction de sous-chaînes

e. Recherche de position

Pour retrouver la position d'une sous-chaîne **`sch`** dans une chaîne **`ch`**, en algorithme, on utilise la fonction **`pos(sch, ch)`**, qui retourne la position de la première occurrence de **`sch`** dans **`ch`**, ou -1 si elle est introuvable.

En Python, on fait appel à : **`ch.find(sch)`**. Cette méthode fonctionne exactement comme la fonction **`pos(sch, ch)`**.

On peut également tester l'existence (l'appartenance) d'une sous-chaîne **`sch`** dans une autre chaîne **`ch`**. En algorithme, on écrit **`sch`** ∈ **`ch`**, qui est traduite en **`sch in ch`**, en Python. Cette expression renvoie un booléen : `True` / `False`.

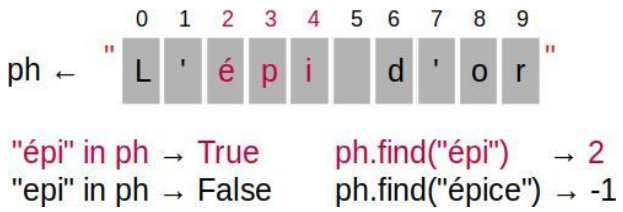

*Figure 5: Exemple d'utilisation de l'opérateur **sch in ch** et la méthode **ch.find(sch)***

La figure 5 illustre la différence entre la méthode **`ch.find(sch)`** et l'opérateur d'existence **`sch in ch`**.

Exemples :

Algorithme	Python

`//  0123456789`
`ph ← "L'épi d'or"`
`p ← pos("épi", ph)  // → 2`
`p ← pos("Randa", ph) // → -1`
	`#  0123456789`
	`ph = "L'épi d'or"`
	`p = ph.find("épi")  # → 2`
	`p = ph.find("Randa") # → -1`

`ex ← "épi" ∈ ph  // → Vrai` `ex ← "Randa" ∈ ph  // → Faux`	`ex = "épi" in ph  # → True` `ex = "Randa" in ph  # → False`

Tableau 7: Exemple de recherche de position et d'appartenance dans une chaîne de caractères

f. Effacer une sous-chaîne

Pour **effacer une partie d'une chaîne** en algorithme, on utilise la fonction **`effacer(ch,deb,fin)`**. Cette fonction retourne le reste de la chaîne après suppression de la partie comprise entre les indices **`deb`** (inclus) et **`fin`** (non inclus) dans une chaîne **`ch`**.

La langage Python ne propose pas une traduction directe de la fonction **`effacer(ch,deb,fin)`**, elle est souvent traduite comme : **`ch[:deb] + ch[fin:]`**. La figure 6 présente un exemple en Python pour émuler la fonction **`effacer()`**.

*Figure 6: Exemple de suppression d'une partie d'une chaîne, ici on veut supprimer les caractères **"at"***

Exemples :

Algorithme	Python

`//  0123456`
`ph ← "Bateaux"`
`mot ← effacer(ph, 1, 3) // → "Beaux"`
	`#  0123456`
	`ph ← "Bateaux"`
	`mot ← ph[:1] + ph[3:] # → "Beaux"`

Tableau 8: Exemple de suppression depuis une chaîne de caractères

g. conversion

En algorithme, une **chaîne** qui contient une **valeur numérique** peut-être transformée en un **entier** ou en un **réel** à l'aide de la fonction **`valeur(chaine)`**. En Python, la fonction prédéfinie **`int(chaine)`** est utilisée lorsque la **`chaine`** contient un entier ou la fonction **`float(chaine)`** si elle contient un réel.

De même, un **entier** ou un **réel** peut être convertit, en algorithme, en une **chaîne** à l'aide de la fonction **`convch(num)`**. Son équivalent, en Python, est la fonction **`str(num)`**.

La figure 7 illustre le processus de conversion.

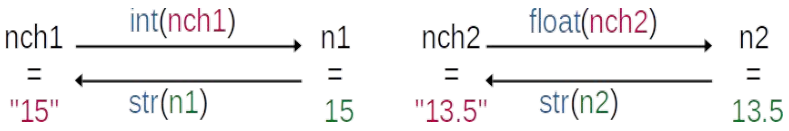

Figure 7: Exemple de conversion d'une chaîne en un nombre ou l'inverse

Exemples :

Algorithme	Python

`nch1 ← "15"`
`n1 ← valeur(nch1) // → 15`
`nch1 ← convch(n1) // → "15"`
	`nch1 = "15"`
	`n1 = int(nch1) # → 15`
	`nch1 = str(n1) # → "15"`

`nch2 ← "13.5"`
`n2 ← valeur(nch2) // → 13.5`
`nch2 ← convch(n2) // → "13.5"`
	`Nch2 = "13.5"`
	`n2 = float(nch2) # → 13.5`
	`nch2 = str(n2)  # → "13.5`

Tableau 9: Exemple de suppression depuis une chaîne de caractères

h. Changement de casse

En algorithme, il est uniquement possible de convertir une chaîne contenant des lettres minuscules en des lettres majuscules à l'aide de la fonction présentée précédemment : **`majus(chaine)`**.

Python, offre lui une variété de méthodes pour changer la casse d'une chaîne de caractère, dont quelques-unes sont présentées dans la figure 8.

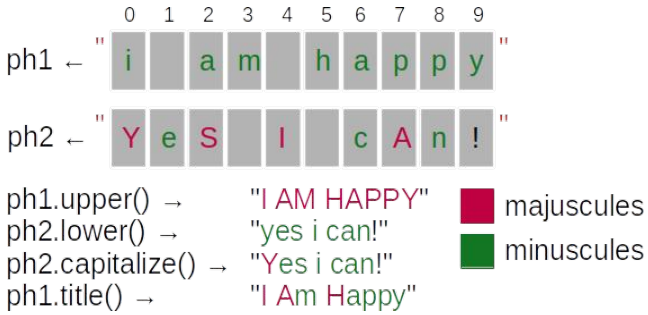

Figure 8: Exemple de changement de la casse d'une chaîne de caractères en Python

3. Type entier

Le type **entier** couvre une partie de l'ensemble des entiers relatifs ℤ, c'est à dire l'ensemble des entiers positifs 0, 1, 2, 3, etc. et des entiers négatifs -1, -2, -3, etc. Les opérateurs possibles avec les entiers sont :

Opérateur	Algorithme	Python
Addition	v1 + v2	v1 + v2
Soustraction	v1 – v2	v1 – v2
Multiplication	v1 * v2	v1 * v2
Division réelle	v1 / v2	v1 / v2
Division entière (quotient)	v1 div v2	v1 // v2
Division entière (reste)	v1 mod v2	v1 % v2
Exponentiation	-	v1 ** v2

Tableau 10: Opérateurs sur les entiers

Fonctions usuelles

Fonction	Rôle	Exemple
Algorithme & Python **abs(v)** -> entier	Retrouver la valeur absolue de **v**.	x1 ← abs(9) // 9 → x1 x2 ← abs(-6) // 6 → x2

Algorithme
aléa(a, b)
-> entier
 	
Python
randint(a, b)
-> int
	Retourner une valeur au hasard dans l'intervalle [**a**, **b**], bornes incluses.	// En algorithme
		de ← aléa(1, 6) // 3 → de
		
		# En Python
		from random import randint
		de = randint(1, 6) # 3 → de

Tableau 11: Fonctions usuelles utilisées avec les entiers

4. Type réel

Le type réel couvre une partie de l'ensemble des réels ℝ, c'est à dire les nombres à virgule flottante. Les opérateurs possibles avec les réels sont les mêmes que ceux disponibles avec les entiers, voir opérateurs avec les entiers.

Fonctions usuelles

Fonction	Rôle	Exemple
Algorithme & Python **abs(v)** -> réel	Retrouver la valeur absolue de **v**.	x1 ← abs(9.2) // 9.2 → x1 x2 ← abs(-6.5) // 6.5 → x2

Algorithme
arrondi(v)
-> entier
 	
Python
round(v)
-> int
	Arrondi **v** à l'entier le plus proche. Si deux multiples sont équidistants, l'arrondi se fait vers la valeur paire.	// En algorithme
		x0 ← arrondi(1.49)// 1 → x0
		x1 ← arrondi(1.5) // 2 → x1
		x2 ← arrondi(1.51)// 2 → x2
		x3 ← arrondi(2.5) // 2 → x3
		# En Python
		x0 = round(1.49) # 1 → x0
		x1 = round(1.5) # 2 → x1
		x2 = round(1.51) # 2 → x2
		x3 = round(2.5) # 2 → x3

Algorithme
Tronc(v)
-> entier
 	
Python
Trunc(v)
-> int
	Supprime la partie décimale de **v**.	// En algorithme
		x0 ← Tronc(1.49)// 1 → x0
		
		# En Python
		from math import trunc
		x0 = trunc(1.49) # 1 → x0

Algorithme
racine(v)
-> réel
 	
Python
sqrt(v)
-> float
	Calcule la racine carré d'un nombre positif **v**.	// En algorithme
		x0 ← racine(25) // 5 → x0
		x1 ← racine(-5) // Erreur
		# En Python
		from math import sqrt
		x0 = sqrt(25) # 5 → x0
		x1 = sqrt(-5) # ValueError

Tableau 12: Fonctions usuelles utilisées avec les entiers

5. Type booléen

Les variables de ce type peuvent prendre uniquement les deux valeurs **`True`** ou **`False`**.

On utilise les **opérateurs de comparaison** pour comparer des valeurs de mêmes types ou de types compatibles :

Algorithme	Python	Description
`<`		Inférieur
`≤`	`<=`	Inférieur ou égal
`>`		Supérieur
`≥`	`>=`	Supérieur ou égal
`=`	`==`	égal
`≠`	`!=`	Différent

Tableau 13: opérateurs de comparaison

On utilise les **opérateurs logiques** pour construire des expressions booléennes plus sophistiquées.

Type	Algorithme	Python	Description
`Unaire`	`non a`	`not a`	**Négation logique**
`Binaire`	`a et b`	`a and b`	**Et logique**
	`a ou b`	`a or b`	**Ou logique**

Tableau 14: opérateurs logiques

Le tableau 15 présente les **tables de vérités** des opérateurs logiques disponibles.

Opérateur unaire

a	non a
False	True
True	False

Opérateurs binaires

a	b	a and b	a or b
False	False	False	False
True	False	False	True
False	True	False	True
True	True	True	True

Tableau 15: tables de vérités des opérateurs logiques

Mise en garde

- Pour les **entiers** et les **réels,** a est inférieur à b (**`a < b`**) si et seulement si **`a - b < 0`**.

  De même, a est égal à b (**`a = b`**) si et seulement si **`a - b = 0`**.

  Aussi, a est différent de b (**`a ≠ b`**) si et seulement si **`a - b ≠ 0`**.

- Pour les **caractères**, la comparaison entre les caractères est basée sur leurs codes ASCII/Unicode.

  `" "` < **`"!"`** < … < **`"0"`** < **`"1"`** < … < **`"A"`** < **`"B"`** < … < **`"a"`** < **`"b"`** < …

- Pour les **chaînes de caractères**, on applique le même principe avec chacune des lettres des deux chaînes.

  `"Amine" < "Eya"` puisque `"A" < "E"` selon leurs codes ASCII.

  `"bassine" < "bassins"` sont de même longueur et commencent par les même lettres, mais le dernier caractère `"e" < "s"` selon leurs codes ASCII.

Module 4 : Structures conditionnelles

1. Forme réduite

Une **structure conditionnelle simple à forme réduite** admet **<u>un seul traitement</u>** qui sera exécuté uniquement si la *condition est vraie*.

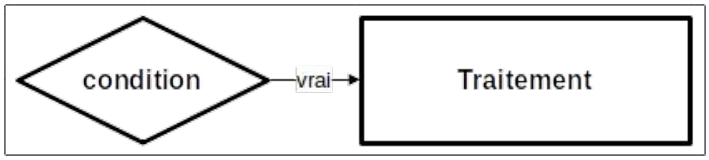

Figure 9: structure conditionnelle simple à forme réduite

Algorithme	Python

Si condition **Alors**
  traitements
Fin Si
	if condition:
	  traitements

Tableau 16: Syntaxe de la forme réduite

Exemples :

Algorithme	Python

// Valeur absolue d'un nombre x
absx ← x
Si x < 0 Alors
  absx ← -x
Fin Si
Ecrire("abs(", x, ")=", absx)
	# Valeur absolue d'un nombre x
	absx = x
	if x < 0:
	  absx = -x
	
	print("abs(", x, ")=", absx)

Tableau 17: Exemple de la forme réduite

2. Forme alternative

Une **structure conditionnelle à forme alternative** admet **<u>deux traitements différents</u>** :

- Le <u>premier traitement</u> est exécuté uniquement si la *condition est vraie,*
- Le <u>second traitement</u> est exécuté uniquement si la *condition est fausse*.

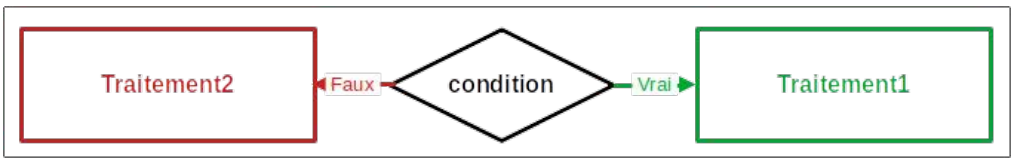

Figure 10: structure conditionnelle simple à forme alternative

Algorithme	Python

Si condition **Alors**
  traitement1
Sinon
  traitement2
Fin Si
	if condition:
	  traitement1
	else:
	  traitement2

Tableau 18: Syntaxe de la forme alternative

Exemples :

Algorithme	Python

`// Maximum de deux nombres`
`Si a > b Alors`
`  max = a`
`Sinon`
`  max = b`
`Fin Si`
`Ecrire("max=", max)`
	`# Maximum de deux nombres`
	`if a > b:`
	`  max = a`
	`else:`
	`  max = b`
	
	`print("max=", max)`

Tableau 19: Syntaxe de la forme alternative

3. Forme généralisée

Une **structure conditionnelle simple à forme généralisée** admet trois traitements différents ou plus qui seront exécutés en fonction de plusieurs conditions.

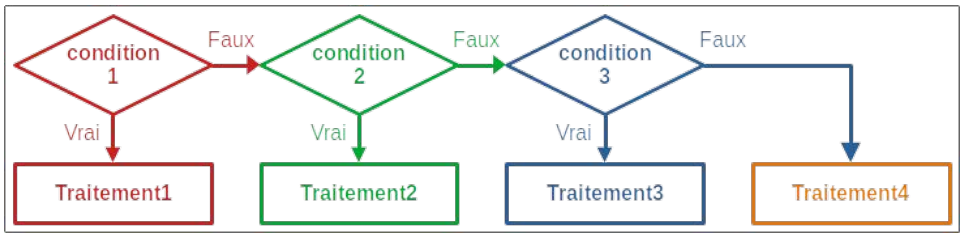

Figure 11: structure conditionnelle simple à forme généralisée

Algorithme	Python

`Si` `condition1` **`Alors`**
`  traitement1`
`Sinon Si` `condition2` **`Alors`**
`  traitement2`
`Sinon`
`  traitement3`
`Fin Si`
	`if` `condition:`
	`  traitement1`
	`elif` `condition2:`
	`  traitement2`
	`else:`
	`  traitement3`

Tableau 20: Syntaxe de la forme généralisée

Exemples :

Algorithme	Python

`// Comparaison entre deux nombres`
`Si a > b Alors`
`  eq ← ">"`
`Sinon Si a < b Alors`
`  eq ← "<"`
`Sinon`
`  eq ← "="`
`Fin Si`
`Ecrire(a, eq, b)`
	`# Comparaison entre deux nombres`
	`if a > b:`
	`  eq = ">"`
	`elif a < b:`
	`  eq = "<"`
	`else:`
	`  eq = "="`
	
	`print(a, eq, b)`

Tableau 21: Exemple de la forme généralisée

Module 5 : Structures itératives (boucles)

1. Boucle pour

Une **structure itérative complète** est utilisée pour **répéter une suite d'instructions un nombre fini de fois** <u>**connu à l'avance**</u>.

Cette structure est basée sur une variable **compteur `cpt`**, voir tableau 15,de type **scalaire**, qui possède initialement une valeur **`deb`**, et qui sera incrémenté (p > 0), ou décrémenté (p < 0) par un **`pas=p`** jusqu'à atteindre ou dépasser la valeur **`fin`**.

Algorithme	Python

Pour cpt **de** Vi **à** Vf **Faire** [Pas=Vp]
  // Traitements
Fin Pour
	for cpt **in** range(Vi, Vf+1, Vp):
	  # Traitements

*Tableau 22: Syntaxe générale de la boucle **pour***

Exemples :

Algorithme	Python

// Compter de 0 à 9
Pour cpt de 0 à 9 Faire
  Ecrire(cpt)
Fin Pour
// -> 0 1 2 3 4 5 6 7 8 9
	# méthode 1 : Compter de 0 à 9
	for cpt in range(10):
	  print(cpt)
	# -> 0 1 2 3 4 5 6 7 8 9
	# méthode 2
	for cpt in range(0, 10):
	  print(cpt)
	# méthode 3
	for cpt in range(0, 10, 1):
	  print(cpt)

// Compter de 9 à 0
Pour cpt de 9 à 0 Faire [pas=-1]
  Ecrire(cpt)
Fin Pour
// -> 9 8 7 6 5 4 3 2 1 0
	# Compter de 0 à 9
	for cpt in range(9, -1, -1):
	  print(cpt)
	
	# -> 9 8 7 6 5 4 3 2 1 0

// Compter de 0 à 9 par pas de 3
Pour cpt de 0 à 9 Faire [pas=3]
  Ecrire(cpt)
Fin Pour
// -> 0 3 6 9
	# Compter de 0 à 9 par pas de 3
	for cpt in range(0, 10, 3):
	  print(cpt)
	
	# -> 0 3 6 9

// Compter de 9 à 0 par pas de 3
Pour cpt de 9 à 0 Faire [pas=-3]
  Ecrire(cpt)
Fin Pour
// -> 9 6 3 0
	# Compter de 9 à 0 par pas de -3
	for cpt in range(9, -1, -3):
	  print(cpt)
	
	# -> 9 6 3 0

Tableau 23: Exemples de la boucle Pour

2. Boucle Répéter ... Jusqu'à

Une s**tructure itérative à condition d'arrêt** est utilisée pour **répéter** une suite d'actions **jusqu'à** ce qu'une **condition** soit <u>vraie</u>.

L'action, figurant dans le corps de la boucle, « **Répéter … Jusqu'à** » est exécutée jusqu'à atteindre une condition. Cette **condition** est représentée par une expression booléenne. Si la condition est évaluée à vraie la boucle est interrompue, sinon on réitère une seconde fois.

Algorithme	Python

Répéter
  // Traitement
Jusqu'à expr_logique
	# Aucune équivalence directe
	condition = False
	while not condition:
	  # traitement
	  condition = expr_logique

*Tableau 24: Syntaxe générale de la boucle **répéter … jusqu'à***

Exemples :

Algorithme	Python

// **Méthode 1**: Compter de 0 à 9
cpt ← 0
Répéter
  Ecrire(cpt)
  cpt ← cpt + 1
Jusqu'à cpt > 9
// -> 0 1 2 3 4 5 6 7 8 9
// **Méthode 2**: Compter de 0 à 9
cpt ← -1
Répéter
  cpt ← cpt + 1
  Ecrire(cpt)
Jusqu'à cpt = 9
// -> 0 1 2 3 4 5 6 7 8 9
	# **Méthode 1** : Compter de 0 à 9
	cpt = 0
	while cpt <= 9:
	  print(cpt)
	  cpt += 1
	# -> 0 1 2 3 4 5 6 7 8 9
	
	# **Méthode 2** : Compter de 0 à 9
	cpt = -1
	while cpt != 9:
	  cpt += 1
	  print(cpt)
	# -> 0 1 2 3 4 5 6 7 8 9

// Saisie d'un entier compris
// entre 3 et 10
Répéter
  Ecrire("Donner n ? ") ; Lire(n)
Jusqu'à 3 ≤ n ≤ 10
	# Saisie d'un entier compris
	# entre 3 et 10
	valid = False
	while not valid:
	  n = int(input("Donner n ? "))
	  valid = 3 <= n <= 10

Tableau 25: Exemples de la boucle répéter … jusqu'à

2. Boucle Tant Que

Une **structure itérative à condition de marche** est utilisée pour **répéter** une suite d'actions **tant que** une **condition** est vraie. L'action, figurant dans le corps de la boucle, « **TantQue** » est exécutée tant qu'une condition est encore valable. Cette **condition** est représentée par une expression booléenne. Si la condition est évaluée à vraie on réitère une seconde fois, sinon la boucle est interrompue.

Algorithme	Python

TantQue condition Faire
  // Traitement
Fin TantQue
	while condition:
	  # traitement

*Tableau 26: Syntaxe générale de la boucle **Tant Que***

Exemples :

Algorithme	Python

`// Calculer le PGCD de deux entiers`
`TantQue a ≠ b Faire`
`  Si a > b Alors`
`  a ← a – b`
`  Sinon`
`  b ← b – a`
`  Fin Si`
`Fin TantQue`
	`# Calculer le PGCD de deux entiers`
	`while a != b:`
	`  if a > b:`
	`  a = a – b`
	`  else:`
	`  b = b – a`

*Tableau 27: Exemples de la boucle **Tant Que***